

A view of 0 or 1 elements: `views::maybe`

Document #: P1255R5
Date: 2019-10-08
Project: Programming Language C++
LEWG
Reply-to: Steve Downey
<sdowney2@bloomberg.net, sdowney@gmail.com>

Abstract: This paper proposes `views::maybe` a range adaptor that produces a view with cardinality 0 or 1 which adapts nullable types such as `std::optional` and pointer to object types.

§ 1 Changes

§ 1.1 Changes since R4

- Use `std::unwrap_reference`
- Remove conditional `noexceptness`
- Adopted the great concept renaming

§ 1.2 Changes since R3

- Always Capture
- Support `reference_wrapper`

§ 1.3 Changes since R2

- Reflects current code as reviewed
- Nullable concept specification

Remove `Readable` as part of the specification, use the useful requirements from `Readable`

- Wording for `views::maybe` as proposed
- Appendix A: wording for a `view_maybe` that always captures

§ 1.4 Changes since R1

- Refer to `views::all` Behavior of capture vs refer is similar to how `views::all` works over the expression it is given

— Use wording ‘range adaptor object’ Match current working paper language

§ 1.5 Changes since R0

— Remove customization point objects

Removed `views::maybe_has_value` and `views::maybe_value`, instead requiring that the nullable type be dereferenceable and contextually convertible to `bool`.

— Concept `Nullable`, for exposition

Concept `Nullable`, which is `Readable` and contextually convertible to `bool`

— Capture rvalues by decay copy

Hold a copy when constructing a view over a nullable rvalue.

— Remove `maybe_view` as a specified type

Introduced two exposition types, one safely holding a copy, the other referring to the nullable

§ 2 Before / After Table

Before	After
<pre> { auto&& opt = possible_value(); if (opt) { // a few dozen lines ... use(*opt); // is *opt OK ? } }</pre>	<pre> for (auto&& opt : views::maybe(possible_value())) { // a few dozen lines ... use(opt); // opt is OK }</pre>
<pre> std::optional o{7}; for (auto&& i : views::maybe(std::ref(o))) { i = 9; std::cout << "i=" << i << " prints 9\n"; } std::cout << "o=" << *o << " prints 9\n";</pre>	<pre> std::optional o{7}; if (o) { *o = 9; std::cout << "o=" << *o << " prints 9\n"; } std::cout << "o=" << *o << " prints 9\n";</pre>
<pre> std::vector<int> v{2, 3, 4, 5, 6, 7, 8, 9, 1}; auto test = [](int i) -> std::optional<int> {</pre>	<pre> std::vector<int> v{2, 3, 4, 5, 6, 7, 8, 9, 1}; auto test = [](int i) -> std::optional<int> {</pre>

Before	After
<pre> switch (i) { case 1: case 3: case 7: case 9: return i; default: return {}; } auto&& r = v ranges::views::transform(test) ranges::views::filter([](auto x){return bool(x);}) ranges::views::transform([] (auto x){return *x;}) ranges::views::transform([](int i) { std::cout << i; return i; }); </pre>	<pre> switch (i) { case 1: case 3: case 7: case 9: return i; default: return {}; } auto&& r = v ranges::views::transform(test) ranges::views::transform(views::maybe) ranges::views::join ranges::views::transform([](int i) { std::cout << i; return i; }); </pre>

§ 3 Motivation

In writing range transformation it is useful to be able to lift a nullable value into a view that is either empty or contains the value held by the nullable. The adapter `views::single` fills a similar purpose for non-nullable values, lifting a single value into a view, and `views::empty` provides a range of no values of a given type. A `views::maybe` adaptor also allows nullable values to be treated as ranges when it is otherwise undesirable to make them containers, for example `std::optional`.

```

std::vector<std::optional<int>> v{
    std::optional<int>{42},
    std::optional<int>{},
    std::optional<int>{6 * 9}};

auto r = views::join(views::transform(v, views::maybe));

for (auto i : r) {
    std::cout << i; // prints 42 and 54
}

```

In addition to range transformation pipelines, `views::maybe` can be used in range based for loops, allowing the nullable value to not be dereferenced within the body. This is of small value in small examples in contrast to testing the nullable in an if statement, but with longer bodies the dereference is often far away from the test. Often the first line in the body of the if is naming the dereferenced nullable, and lifting the dereference into the for loop eliminates some boilerplate code, the same way that range based for loops do.

```
{
    auto&& opt = possible_value();
    if (opt) {
        // a few dozen lines ...
        use(*opt); // is *opt OK ?
    }
}

for (auto&& opt : views::maybe(possible_value())) {
    // a few dozen lines ...
    use(opt); // opt is OK
}
```

The view can be on a `std::reference_wrapper`, allowing the underlying nullable to be modified:

```
std::optional o{7};
for (auto&& i : views::maybe(std::ref(o))) {
    i = 9;
    std::cout << "i=" << i << " prints 9\n";
}
std::cout << "o=" << *o << " prints 9\n";
```

Of course, if the nullable is empty, there is nothing in the view to modify.

```
auto oe = std::optional<int>{};
for (int i : views::maybe(std::ref(oe)))
    std::cout << "i=" << i << '\n'; // does not print
```

Converting an optional type into a view can make APIs that return optional types, such a lookup operations, easier to work with in range pipelines.

```
std::unordered_set<int> set{1, 3, 7, 9};

auto flt = [=](int i) -> std::optional<int> {
    if (set.contains(i))
        return i;
    else
        return {};
};

for (auto i : ranges::iota_view{1, 10} |
ranges::views::transform(flt)) {
    for (auto j : views::maybe(i)) {
        for (auto k : ranges::iota_view(0, j))
            std::cout << 'a';
        std::cout << '\n';
    }
}
```

```
    }  
}  
  
// Produce 1 ring, 3 rings, 7 rings, and 9 rings
```

§ 4 Proposal

Add a range adaptor object `views::maybe`, returning a view over a nullable object, capturing by value temporary nullables. A `Nullable` object is one that is both contextually convertible to `bool` and for which the type produced by dereferencing is an equality preserving object. Non void pointers, `std::optional`, and the proposed outcome and expected types all model `Nullable`. Function pointers do not, as functions are not objects. Iterators do not generally model `Nullable`, as they are not required to be contextually convertible to `bool`.

§ 5 Design

The basis of the design is to hybridize `views::single` and `views::empty`. If the underlying object claims to hold a value, as determined by checking if the object when converted to `bool` is true, begin and end of the view are equivalent to the address of the held value within the underlying object and one past the underlying object. If the underlying object does not have a value, begin and end return `nullptr`.

§ 6 LEWG Attention

Call LEWG's attention to the use of `ptrdiff_t` as the return type of `size` (which is consistent with `single_view`). The author has a weak preference for a signed type here, but a strong preference for consistency with other Range types.

Call LEWG's attention to removing the conditional `noexcept` constructors, consistent with `single_view`.

§ 7 Synopsis

§ 7.1 Maybe View

`views::maybe` returns a View over a `Nullable` that is either empty if the nullable is empty, or provides access to the contents of the nullable object.

The name `views::maybe` denotes a range adaptor object (`[range.adaptor.object]`). For some subexpression `E`, the expression `views::maybe(E)` is expression-equivalent to:

- `maybe_view{E}`, the View specified below, if the expression is well formed, where `decay-copy(E)` is moved into the `maybe_view`

— otherwise `views::maybe(E)` is ill-formed.

[Note: Whenever `views::maybe(E)` is a valid expression, it is a prvalue whose type models `View`. — end note]

§ 7.2 Concept *nullable*

Types that:

- are contextually convertible to `bool`
- are dereferenceable
- have const references which are dereferenceable
- the `iter_reference_t` of the type and the `iter_reference_t` of the const type, will :
 - satisfy `is_lvalue_reference`
 - satisfy `is_object` when the reference is removed
 - for const pointers to the referred to types, satisfy `ConvertibleTo` model the exposition only `Nullable` concept
- Or are a `reference_wrapper` around a type that satisfies `Nullable`

Given a value `i` of type `I`, `I` models `nullable` only if the expression `*i` is equality-preserving. [Note: The expression `*i` is required to be valid via the exposition-only `nullable` concept). — end note]

For convenience, the exposition-only `is_reference_wrapper_v` is used below.

```
// For Exposition
template <typename T>
struct is_reference_wrapper :std::false_type {};
template <typename T>
struct is_reference_wrapper<std::reference_wrapper<T>>
    :std::true_type {};
template <typename T>
inline constexpr bool is_reference_wrapper_v =
    is_reference_wrapper<T>::value;
```

```
namespace std::ranges {

// For Exposition
template <class Ref, class ConstRef>
concept bool readable_references =
    is_lvalue_reference_v<Ref> &&
    is_object_v<remove_reference_t<Ref>> &&
    is_lvalue_reference_v<ConstRef> &&
    is_object_v<remove_reference_t<ConstRef>> &&
    convertible_to<add_pointer_t<ConstRef>,
        const remove_reference_t<Ref>*>;

template <class T>
```

```

concept bool nullable =
    is_object_v<T> &&
    requires(T& t, const T& ct) {
        bool(ct); // Contextually bool
        *t; // T& is deferenceable
        *ct; // const T& is deferenceable
    }
    && readable_references<iter_reference_t<T>, // Ref
                        iter_reference_t<const T>>; // ConstRef

template <class T>
concept bool wrapped_nullable =
    is_reference_wrapper_v<T>
    && nullable<typename T::type>;

```

§ 7.3 *maybe_view*

```

template <typename Maybe>
requires ranges::copy_constructible<Maybe> &&
(nullable<Maybe> || wrapped_nullable<Maybe>)
class maybe_view
    : public ranges::view_interface<maybe_view<Maybe>> {
private:
    // For Exposition
    using T = /* see below */
        semiregular-box<Maybe> value_;

public:
    constexpr maybe_view() = default;
    constexpr explicit maybe_view(Maybe const& maybe);

    constexpr explicit maybe_view(Maybe&& maybe);

    template<class... Args>
    requires Constructible<Maybe, Args...>
    constexpr maybe_view(in_place_t, Args&&... args);

    constexpr T*      begin() noexcept;
    constexpr const T* begin() const noexcept;
    constexpr T*      end() noexcept;
    constexpr const T* end() const noexcept;

    constexpr std::ptrdiff_t size() const noexcept;

    constexpr T* data() noexcept;
    constexpr const T* data() const noexcept;
};

```

Where the type alias T is the `iter_reference_t` with the reference removed of either the type `Maybe` or the type `reference_wrapper::type`.

```

// For Exposition
using T = std::remove_reference_t<
    ranges::iter_reference_t<typename unwrap_ref<Maybe>::type>>;

```

```
constexpr explicit maybe_view(Maybe const& maybe)
    noexcept(std::is_nothrow_copy_constructible_v<Maybe>);
```

Effects: Initializes value_ with maybe.

```
constexpr explicit maybe_view(Maybe&& maybe)
    noexcept(std::is_nothrow_move_constructible_v<Maybe>);
```

Effects: Initializes value_ with std::move(maybe).

```
template<class... Args>
constexpr maybe_view(in_place_t, Args&&... args);
```

Effects: Initializes value_ as if by value_{in_place, std::forward<Args>(args)...}.

```
constexpr T* begin() noexcept;
constexpr const T* begin() const noexcept;
```

Effects: Equivalent to: return data();.

```
constexpr T* end() noexcept;
constexpr const T* end() const noexcept;
```

Effects: Equivalent to: return data() + size();.

```
static constexpr ptrdiff_t size() noexcept;
```

Effects: Equivalent to:

```
if constexpr (is_reference_wrapper_v<Maybe>) {
    return bool(value_.get().get());
} else {
    return bool(value_.get());
}
```



```
constexpr T* data() noexcept;
```

Effects: Equivalent to:

```
Maybe& m = value_.get();
if constexpr (is_reference_wrapper_v<Maybe>) {
    return m.get() ? std::addressof(*(m.get())) : nullptr;
} else {
    return m ? std::addressof(*m) : nullptr;
}
```

```
constexpr const T* data() const noexcept;
```

Effects: Equivalent to:

```
const Maybe& m = value_.get();
if constexpr (is_reference_wrapper_v<Maybe>) {
    return m.get() ? std::addressof(*(m.get())) : nullptr;
}
```



```
    } else {  
        return m ? std::addressof(*m) : nullptr;  
    }
```

§ 7.4 **view::maybe**

The name `views::maybe` denotes a customization point object ([customization.point.object]). For some subexpression `E`, the expression `views::maybe(E)` is expression-equivalent to `maybe_view{E}`.

§ 8 **Impact on the standard**

A pure library extension, affecting no other parts of the library or language.

§ 9 **References**

[P0896R3] Eric Niebler, Casey Carter, Christopher Di Bella. The One Ranges Proposal
URL: <https://wg21.link/p0896r3>

[P0323R7] Vicente Botet, JF Bastien. `std::expected` URL: <https://wg21.link/p0323r7>